



ΕΡΓΑΣΙΑ 2

ΙΑΤΡΟΥ ΚΩΝΣΤΑΝΤΙΝΟΣ 58071, ΜΙΧΑΛΑΚΗΣ ΑΘΑΝΑΣΙΟΣ 58226, ΜΠΟΥΡΧΑ ΙΩΑΝΝΑ 58019

ΕΡΓΑΣΙΑ 2

Δευτερεύουσες συναρτήσεις

```
def get_secondary_structure(n: int, sorted_pairs)->str:

    s = ['.']*n

    for p in sorted_pairs:
        s[p[0]] = "("
        s[p[1]] = ")"

    return "".join(s)
```

Αρχικά: Πίνακας με τελείες (νουκλεοτίδια)

Αλγόριθμος: $\cdot \rightarrow ($ (Αριστερό μέρος δεσμού)
 $\cdot \rightarrow)$ (Δεξί μέρος δεσμού)

Επιστρέφει: String με dot bracket notation

```
setAU = set('A')
setAU.add('U')
setCG = set('C')
setCG.add('G')
valid_pairs = [setAU, setCG]

def can_pair(base_1:str, base_2:str)->bool:

    pair = set(base_1)
    pair.add(base_2)
    return pair in valid_pairs
```

Αρχικά: Δημιουργία σετ με τις 2 βάσεις

Αλγόριθμος: Έλεγχος αν το σετ υπάρχει στο
valid_pairs

Επιστρέφει: True/False

ΕΡΓΑΣΙΑ 2

Κύρια συνάρτηση

```
def get_opt_val(i:int, j:int)->int:
    if opt_val[i][j] == -1:
        if j - i <= 4:
            opt_val[i][j] = 0
        else:
            best = get_opt_val(i, j-1)
            best_pairs = opt_val_pairs[i][j-1]
            for t in range(i, j-4):
                a = sequence[t]
                d = sequence[j]
                if can_pair(a, d):
                    temp = 1 + get_opt_val(i, t-1) + get_opt_val(t+1, j-1)
                    AddSet = [(t, j)]
                    AddSet.extend(opt_val_pairs[i][t-1])
                    AddSet.extend(opt_val_pairs[t+1][j - 1])
                    if temp > best:
                        best = temp
                        best_pairs = AddSet
            opt_val[i][j] = best
            opt_val_pairs[i][j] = best_pairs

    return opt_val[i][j]
```

```
def get_opt_val(i:int, j:int)->int:
    if opt_val[i][j] == -1:
        if j - i <= 4:
            opt_val[i][j] = 0
        else:
            best = get_opt_val(i, j-1)
            best_pairs = opt_val_pairs[i][j-1]
            for t in range(i, j-4):
                if can_pair(sequence[t], sequence[j]):
                    temp = 1 + get_opt_val(i, t-1) + get_opt_val(t+1, j-1)
                    AddSet = [(t, j)] + opt_val_pairs[i][t-1] + opt_val_pairs[t+1][j - 1]
                    if temp > best:
                        best = temp
                        best_pairs = AddSet
            opt_val[i][j] = best
            opt_val_pairs[i][j] = best_pairs

    return opt_val[i][j]
```



Υπάρχει λύση → Επιστρέφει την λύση

Αλλιώς:

Αρχικά την λύση της υπο-αλυσίδας $(i, j-1)$

Υπολογίζει όλες τις δυνατές λύσεις αν υπάρχει δυνατότητα δεσμού για το j

Αν η λύση είναι καλύτερη την κρατάει

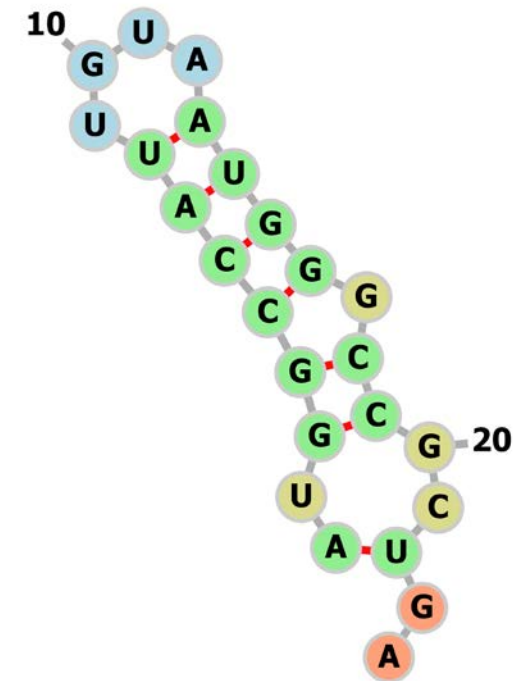


ΕΡΓΑΣΙΑ 2

Αποτελέσματα 1

Ακολουθία: **AUGGCCAUUGUAAUGGGCCGCUGA**

Dot-Bracket Notation: **(.(((((((....)))))..))..)**



ΕΡΓΑΣΙΑ 2

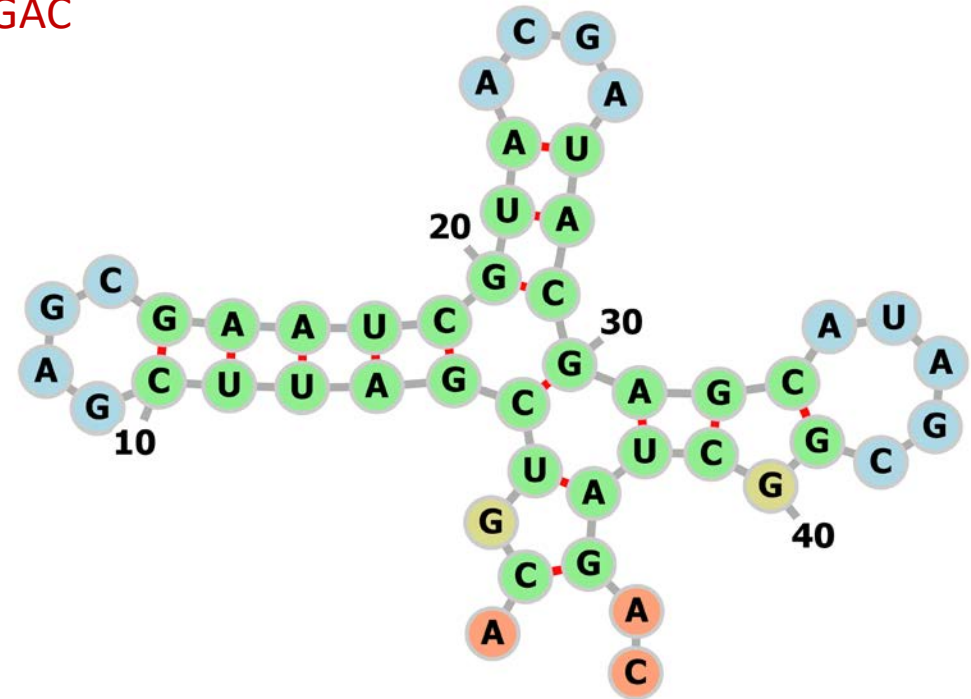
Αποτελέσματα 2

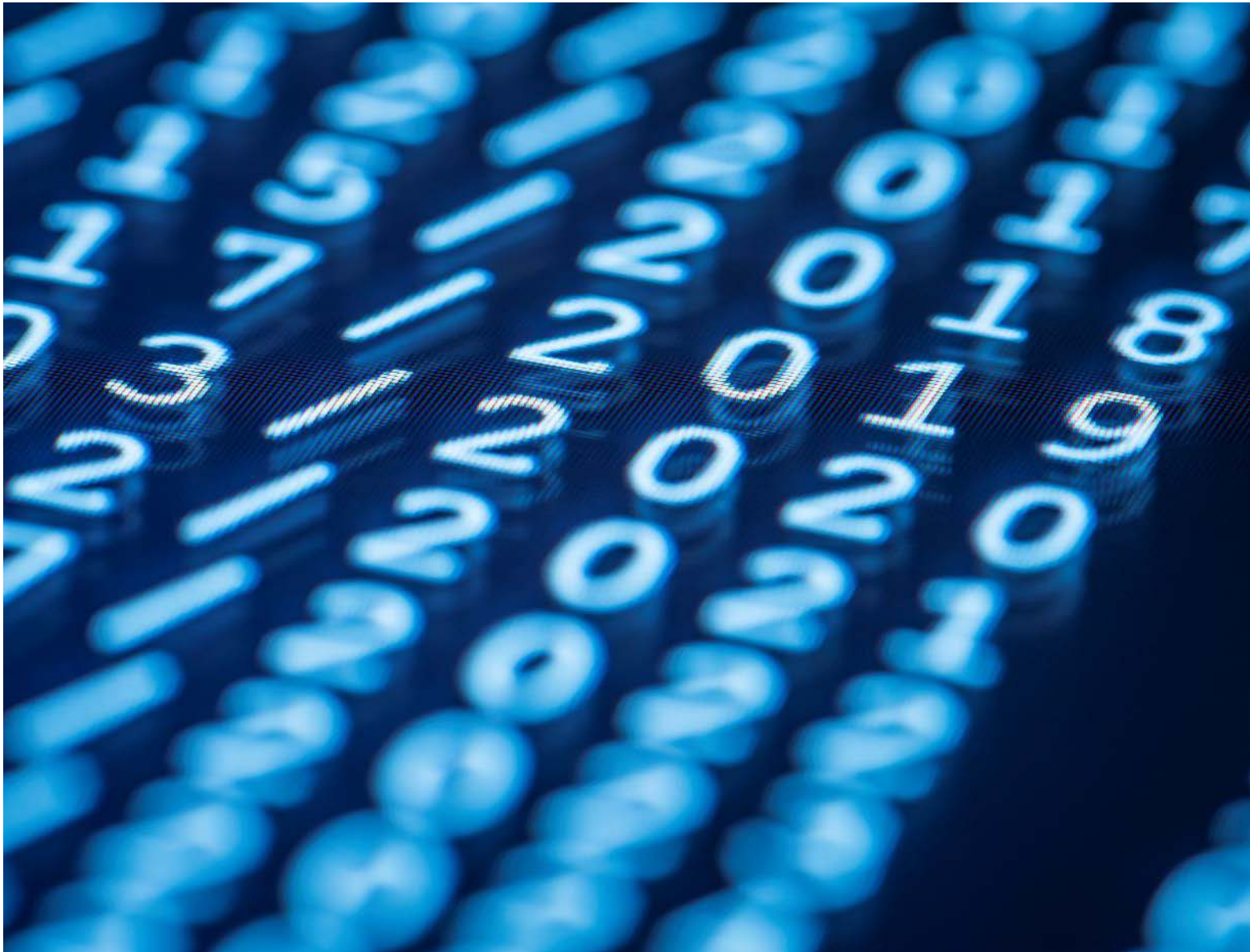
Ακολουθία:

ACGUCGAUUCGAGCGAAUCGUAACGAUACGAGCAUAGCGGCUAGAC

Dot-Bracket Notation:

.(.(((((((.....))))))(((.....))))(((.....).))))..





ΕΥΧΑΡΙΣΤΟΥΜΕ